

6312 Петров.В Отчет

В ходе лабораторной работы были сгенерированы квадратные матрицы 100 на 100 при помощи matrix.py. Матрицы находятся в файлах matrixA.txt и matrixB.txt.

```
PS D:\nn\lab1> & c:\Users\padim\AppData\Local\Programs\Python\Python313\python.exe d:/nn/lab1/matrix.py
Matrices generated
```

При помощи mult.cpp было выполнено перемножение матриц и результат записан в result.txt

```
Matrix size: 100x100
Operations: 2000000
Execution time: 0.009345 seconds
```

Далее была проведена верификация результата перемножения при помощи verify.py

```
Verification SUCCESS
PS D:\nn\lab1> █
```

Эксперименты

Размер	Количество операций	Время
100x100	2000000	0.009345
200x200	16000000	0.0794361
400x400	128000000	0.788852
800x800	1024000000	8.32356
1200x1200	3456000000	38.1938
1600x1600	8192000000	95.2164
2000x2000	16000000000	183.518

Исходный код программы:

matrix.py

```
import numpy as np

n = 100

A = np.random.randint(0, 10, (n, n))
B = np.random.randint(0, 10, (n, n))

def save_matrix(filename, M):
    with open(filename, "w") as f:
        f.write(str(len(M)) + "\n")
        for row in M:
            f.write(" ".join(map(str, row)) + "\n")

save_matrix("matrixA.txt", A)
save_matrix("matrixB.txt", B)

print("Matrices generated")
```

mult.cpp

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>

using namespace std;
```

```

vector<vector<double>> readMatrix(string filename, int fin)
{
    ifstream file(filename);
    file >> n;

    vector<vector<double>> M(n, vector<double>(n));

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            file >> M[i][j];

    return M;
}

void writeMatrix(string filename, vector<vector<double>> fiM)
{
    ofstream file(filename);

    int n = M.size();

    file << n << endl;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            file << M[i][j] << " ";
        file << endl;
    }
}

vector<vector<double>> multiply(vector<vector<double>> fiA,
                               vector<vector<double>> fiB)
{
    int n = A.size();

    vector<vector<double>> C(n, vector<double>(n, 0));

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            for(int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];

    return C;
}

int main()
{
    int n1, n2;

    auto A = readMatrix("matrixA.txt", n1);
    auto B = readMatrix("matrixB.txt", n2);

    if(n1 != n2)
    {
        cout << "Matrix sizes are different" << endl;
        return 1;
    }
}

```

```

int n = n1;

auto start = chrono::high_resolution_clock::now();

auto C = multiply(A, B);

auto end = chrono::high_resolution_clock::now();

chrono::duration<double> time = end - start;

writeMatrix("result.txt", C);

long long operations = 2ll * n * n * n;

cout << "Matrix size: " << n << "x" << n << endl;
cout << "Operations: " << operations << endl;
cout << "Execution time: " << time.count() << " seconds" << endl;

return 0;
}

```

verify.py

```

import numpy as np

def read_matrix(filename):
    with open(filename) as f:
        n = int(f.readline())
        data = []

        for _ in range(n):
            data.append(list(map(float, f.readline().split())))

    return np.array(data)

A = read_matrix("matrixA.txt")
B = read_matrix("matrixB.txt")
matrix_cpp = read_matrix("result.txt")

matrix_python = A @ B

if np.allclose(matrix_cpp, matrix_python):
    print("Verification SUCCESS")
else:
    print("Verification FAILED")

```

Вывод: в ходе лабораторной работы я изучил механизмы перемножения матриц на языке программирования C++, а также сравнил с библиотекой python результат работы моей программы.